Java Backend Architecture

Interview Series

Chapter 18.8

Nginx Caching

50+ Interview Q&A	Code Examples	Architecture Diagrams	Advanced Topics

Chapter 18.8 – Nginx Caching

Overview

Nginx's proxy cache reduces load on Java backends by storing responses on disk or memory and serving them directly for subsequent identical requests. Correct cache key design, bypass rules for authenticated requests, and cache invalidation strategies are critical to avoid serving stale or private data. Understanding proxy_cache_path, cache keys, and cache status headers is required for senior-level performance optimisation.

Key Definitions

Term	Definition
proxy_cache_path	Defines cache storage: path, directory structure, shared memory zone, size limits.
proxy_cache	Enables caching for a location using a named zone defined by proxy_cache_path.
proxy_cache_key	Defines what constitutes a unique cacheable request. Default includes scheme, host, URI.
proxy_cache_valid	Sets TTL per response code. 200 10m = cache 200 responses for 10 minutes.
proxy_cache_bypass	If variable is non-empty/non-zero, skip cache and fetch from upstream.
proxy_no_cache	If variable is non-empty/non-zero, do not store the response in cache.
$upstream_cache_status	Variable exposing cache result: HIT, MISS, BYPASS, EXPIRED, STALE, UPDATING, REVALIDATED.
proxy_cache_use_stale	Serve stale cached response on specific upstream errors (error, timeout, updating).
proxy_cache_lock	Only one request fetches from upstream for a given cache key; others wait. Prevents thundering herd.
levels	Directory hierarchy depth for cache storage (e.g. 1:2 = two-level directory tree).

Diagram 1: Cache Request Flow

```
Client Request
     |
     v
[Nginx Cache Check]
     |
     +-- HIT? --------> Serve from cache (no upstream call)
     |
     +-- MISS/BYPASS -> proxy_pass to Java backend
                              |
                              v
                         [Java Backend Response]
                              |
                              +-- cacheable? -> Store in cache
                              |
                              v
                         Return to client + add X-Cache-Status header
```

Diagram 2: Cache Directory Structure (levels=1:2)

```
proxy_cache_path /var/cache/nginx levels=1:2 keys_zone=api:10m

Cache key hash: e.g. b7f3a1c9d8e2f4b6...

File stored at:
/var/cache/nginx/
  6/           (last 1 char of hash)
    b7/         (2 chars before last)
      b7f3a1c9d8e2f4b6...  (full hash = cache file)

levels=1:2 avoids single-directory with millions of files
(filesystem performance degrades with too many files per dir)
```

Code Examples

Cache Zone Configuration:

```
http {
    proxy_cache_path /var/cache/nginx
        levels=1:2
        keys_zone=api_cache:10m    # 10MB shared memory for keys (~80k entries)
        max_size=1g                # max disk usage, LRU eviction
        inactive=60m               # evict if not accessed in 60 minutes
        use_temp_path=off;         # write directly (avoids extra copy on SSD)

    server {
        location /api/public/ {
            proxy_cache            api_cache;
            proxy_cache_key        '$scheme$request_method$host$request_uri';
            proxy_cache_valid      200 10m;
            proxy_cache_valid      301 1h;
            proxy_cache_valid      404 1m;
            proxy_cache_valid      any 1m;
            proxy_cache_use_stale  error timeout updating http_500 http_503;
            proxy_cache_lock       on;
            add_header             X-Cache-Status $upstream_cache_status;
            proxy_pass             http://java_api;
        }
    }
}
```

Cache Bypass for Authenticated Requests:

```
location /api/ {
    proxy_cache     api_cache;

    # Bypass and don't cache authenticated or no-cache requests
    proxy_cache_bypass $http_authorization $http_pragma;
    proxy_no_cache     $http_authorization $http_pragma;

    # Bypass for POST/PUT/DELETE (not idempotent)
    proxy_cache_methods GET HEAD;

    add_header X-Cache-Status $upstream_cache_status always;
    proxy_pass http://java_api;
}
```

Cache Purge (ngx_cache_purge module):

```
# Install: --add-module=ngx_cache_purge
location ~ /purge(/.*) {
    allow 10.0.0.0/8;     # restrict to internal
    deny  all;
    proxy_cache_purge api_cache $scheme$host$1;
}

# Usage: PURGE http://nginx-host/purge/api/products/123
# Java backend calls this after updating product data
```

Interview Q&A – Nginx Caching

Q1. What does proxy_cache_key determine?

The cache key uniquely identifies a cacheable response. Default: $scheme$proxy_host$request_uri. Include $request_method to prevent caching POST responses. Include $http_accept_encoding for compressed vs uncompressed variants. Include $cookie_user_tier for per-tier caching. Bad key: missing $host causes responses from api.example.com to be served for app.example.com -- data leak.

Q2. What does $upstream_cache_status show?

HIT: served from cache. MISS: not in cache, fetched from upstream, now stored. BYPASS: cache bypassed (proxy_cache_bypass variable was non-empty). EXPIRED: cache entry expired, fetched fresh. STALE: expired but upstream unavailable, served stale (proxy_cache_use_stale). UPDATING: stale response served while background fetch in progress. REVALIDATED: 304 Not Modified from upstream. Log this for cache hit rate analysis in Grafana.

Q3. How does proxy_cache_lock prevent thundering herd?

Without lock: 1000 simultaneous requests for uncached /api/products all miss the cache simultaneously and all hit the Java backend -- thundering herd. With proxy_cache_lock on: first request fetches from upstream; remaining 999 wait for the first to populate cache, then serve from cache. proxy_cache_lock_timeout 5s: if lock holder doesn't complete in 5s, waiting requests pass through to upstream.

Q4. How does proxy_cache_use_stale work?

proxy_cache_use_stale error timeout updating: if upstream returns error, times out, or is being updated, serve the expired cached response instead of 502/504. Critical for resilience: Java backend deployment or brief outage serves last-known-good cached responses to clients. proxy_cache_background_update on refreshes expired entries in background, eliminating cache miss latency spike on expiry.

Q5. Why set use_temp_path=off in proxy_cache_path?

By default, Nginx writes cached responses to a temp directory first, then renames to cache directory. On different filesystems, rename becomes a copy operation (expensive). use_temp_path=off writes directly to cache directory, eliminating the copy. On SSD storage (typical for Nginx cache), this reduces write latency and disk I/O. Always set off for production deployments.

Q6. How do you implement cache invalidation from a Java backend?

Options: 1) Time-based (proxy_cache_valid TTL) -- simple, no infrastructure needed. 2) Cache purge API (ngx_cache_purge module): Java backend calls PURGE /purge/path after data update. 3) Cache key includes version: proxy_cache_key includes $arg_v=123, Java increments version on update. 4) Nginx Plus: /api/cache purge endpoint. Most practical: short TTLs (30-60s) for frequently changing data.

Q7. What should never be cached?

Never cache: responses with Set-Cookie header (serves another user's cookie), responses with Authorization header in request (private data), POST/PUT/DELETE responses (side-effecting), personalised responses, payment or checkout pages, real-time data (stock prices, live scores). Use proxy_cache_bypass $http_authorization to bypass for authenticated requests. proxy_cache_methods GET HEAD limits to safe methods.

Q8. How does the keys_zone size affect performance?

keys_zone allocates shared memory for cache key lookup table (not the cached content -- that's on disk). 1MB stores ~8k keys; 10MB stores ~80k keys. If zone is full, Nginx evicts LRU entries from disk. Undersized zone causes frequent evictions. Cache metadata lookup is O(1) hash. keys_zone should be sized to hold all expected active cache keys. Monitor with nginx_status or Nginx Plus /api/caches endpoint.

Q9. How do you serve stale while revalidating in background?

proxy_cache_background_update on + proxy_cache_use_stale updating: when a cached entry expires, Nginx serves the stale response immediately while spawning a background request to refresh the cache. Next client request sees fresh data. Eliminates cache miss latency spike at TTL expiry. Essential for high-traffic Java APIs where even one cache miss per key causes upstream spike.

Q10. How does Nginx handle Vary header for cache variants?

Vary: Accept-Encoding (gzip vs non-gzip) and Vary: Accept-Language create separate cache entries per variant. Nginx respects Vary from upstream Java backend. Issue: Vary: * disables caching entirely. Include Accept-Encoding in proxy_cache_key when caching compressed responses: proxy_cache_key '$scheme$host$request_uri$http_accept_encoding'. Excessive Vary variants

fragment cache reducing hit rate.